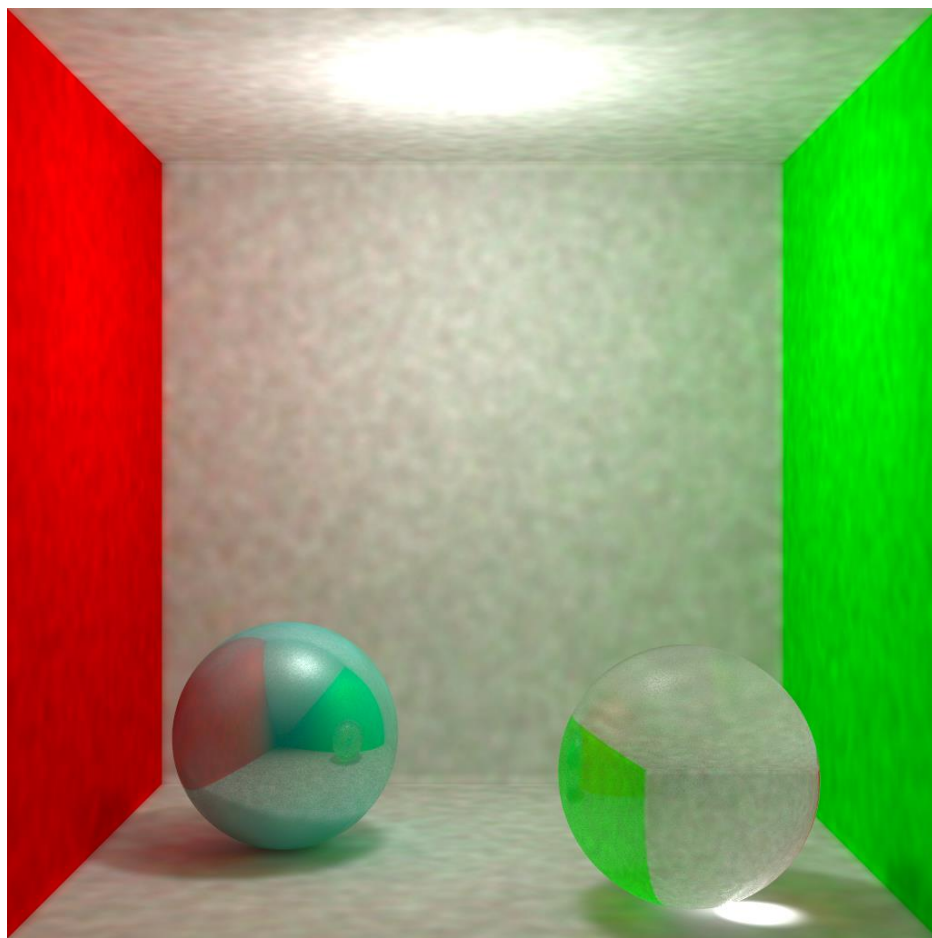


Photon Mapper

INFORAMTICA GRÁFICA (2024-2025)



Alex Petrov

NIP: 812019 | 18/06/2025

Contenido

1. Introducción	2
2. Ecuación de render	2
2.1. Photon Mapping	2
2.2. Iluminación directa	3
3. Escenas.....	3
3.1. Cornell box con bola de plástico y bola de cristal	3
3.2. Cornell box con bolas difusas.....	5
3.3. Convergencia del algoritmo	6
4. Extensiones Implementadas	7
4.1. Otros kernels – kernel gaussiano.....	7
5. Referencias	8

1. Introducción

Este proyecto consiste en la implementación desde cero de un algoritmo de renderizado global basado en Photon Mapping, utilizando C++ y estructuras propias. La técnica de photon mapping permite simular de forma más precisa fenómenos complejos de iluminación como caústicas, color bleeding o reflexión múltiple, que resultan difíciles de representar con un simple path tracer.

Durante el desarrollo se ha diseñado un sistema de representación de la escena capaz de gestionar de forma eficiente la emisión, almacenamiento y consulta de fotones, además de integrar un modelo de materiales basado en BSDFs que incluye materiales difusos, especulares y dieléctricos. También se ha prestado especial atención a la integración con sistemas multihilo, medición de rendimiento y herramientas de visualización que faciliten la evaluación cualitativa y cuantitativa del algoritmo. A lo largo del documento se detalla la implementación, las decisiones técnicas adoptadas y los resultados obtenidos.

2. Ecuación de render

La ecuación de render se define como:

$$L_0(x, \vec{\omega}_0) = L_e(x, \vec{\omega}_0) + \int_{\Omega} L_i(x, \vec{\omega}_i) f_r(x, \vec{\omega}_i, \vec{\omega}_0) |\vec{n} \cdot \vec{\omega}_i| d\vec{\omega}_i$$

Donde:

- L_0 es la luz saliente en un punto x en una dirección $\vec{\omega}_0$
- L_e es la radiación emitida en un punto x en una dirección $\vec{\omega}_0$
- $\int_{\Omega}$ es la integral sobre todas las direcciones posibles ($\vec{\omega}_i$)
- L_i es la luz incidente en un punto x desde una dirección $\vec{\omega}_i$
- $f_r(x, \vec{\omega}_i, \vec{\omega}_0)$ es la función de reflectancia de la superficie, o BRDF (*Bidirectional Reflectance Distribution Function*). Esta define como una superficie refleja la luz incidente, relacionando la dirección de entrada $\vec{\omega}_0$ con la dirección de salida $\vec{\omega}_i$.
- $|\vec{n} \cdot \vec{\omega}_i|$ es el producto escalar entre la dirección de llegada de la luz ($\vec{\omega}_i$) y la normal de la superficie ($\vec{n}$).

2.1. Photon Mapping

El procedimiento de Photon Mapping se divide en dos fases principales. En la primera fase, conocida como emisión de fotones, se lanzan millones de rayos desde las fuentes de luz, simulando el comportamiento de la luz al interactuar con los objetos de la escena. Cada vez que un fotón impacta en una superficie, su información (posición, dirección, energía, color) se almacena en un mapa de fotones, utilizando una estructura eficiente como un KD-tree para permitir consultas rápidas posteriormente. Los rebotes del fotón dependen del tipo de material: puede reflejarse, refractarse o ser absorbido, siguiendo un modelo probabilístico como la ruleta rusa.

En la segunda fase, renderizado, se lanzan rayos desde la cámara y, al detectar intersecciones con la escena, se evalúa la iluminación directa mediante Next Event Estimation y la indirecta consultando el mapa de fotones. Esta aproximación evita el trazado recursivo de múltiples rebotes, y permite capturar efectos globales como caústicas, interreflexiones suaves e iluminación difusa de forma eficiente. Así, Photon Mapping proporciona un equilibrio entre realismo y rendimiento, especialmente útil en escenas con iluminación compleja.

2.2. Iluminación directa

Podemos estimar la iluminación directa de la siguiente manera:

$$L_0(x, \vec{\omega}_0) \approx \sum_{p=1}^k f_r(x, \vec{\omega}_i, \vec{\omega}_0) \frac{\Phi_p}{\pi r_k^2}$$

Donde:

- Φ_p es el flujo del fotón
- r_k es la distancia de los fotones más cercanos
- k es el número de fotones más cercanos

3. Escenas

Para mostrar los resultados que produce el path tracer desarrollado, se mostraran distintos rendes sobre dos escenas:

- Cornell box con paredes difusas, pared roja a la izquierda y verde a la derecha, con dos bolas difusas, una azul y otra rosa.
- Cornell box con paredes difusas, pared roja a la izquierda y verde a la derecha, con dos bolas, una de plástico azul (mix de difuso y especular), y otra bola de cristal (mix de especular y refracción)

Cada imagen se renderiza a una resolución de 256x256 pixeles, y 6 rebotes para cada path del algoritmo de path tracing y los fotones. Se usarán 32 muestras por pixel.

3.1. Cornell box con bola de plástico y bola de cristal

Primero se fijará un numero de vecinos para la estimación de radiancia, $k=10$.

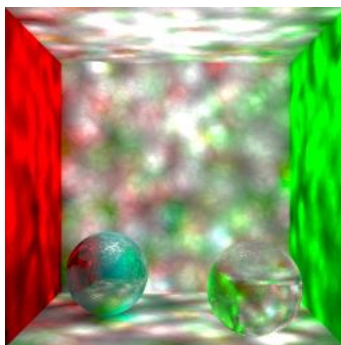


Figura 3.1 – 1K fotones. 10 vecinos

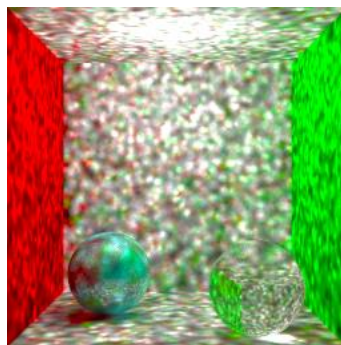


Figura 3.2 - 10K fotones. 10 vecinos

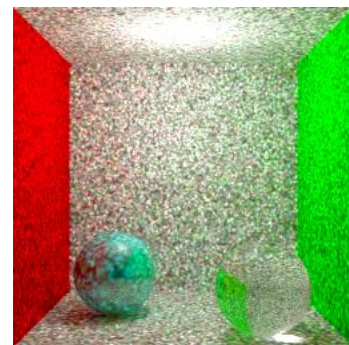


Figura 3.3 - 100K fotones. 10 vecinos

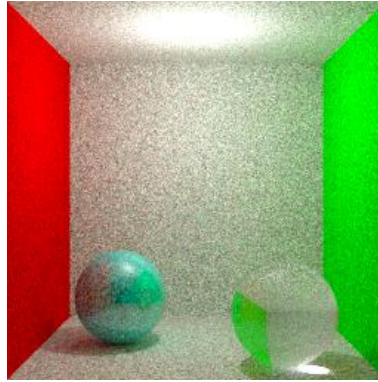


Figura 3.4 - 1M fotones. 10 vecinos

Segundo, fijaremos un número de fotón es, $N=100K$.

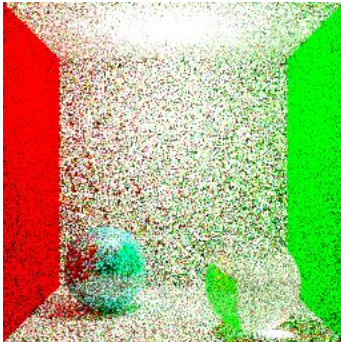


Figura 3.5 - 100K fotones. 1 vecino

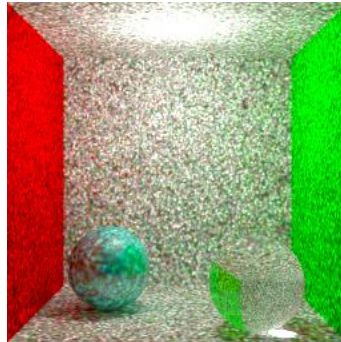


Figura 3.6 - 100K fotones. 10 vecinos

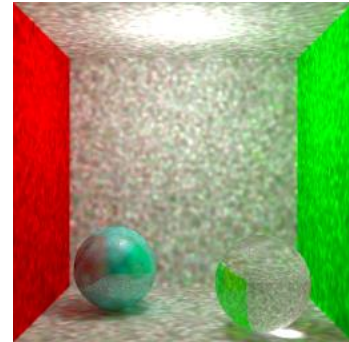


Figura 3.7 - 100K fotones. 50 vecinos

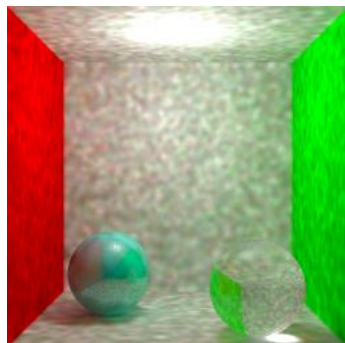


Figura 3.8 - 100K fotones. 100 vecinos

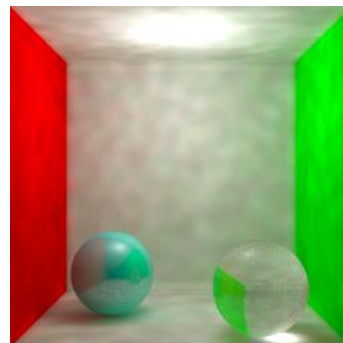


Figura 3.9 - 100K fotones. 500 vecinos

Las imágenes muestran cómo afectan dos parámetros clave del algoritmo de Photon Mapping: el número de fotones almacenados y el número de vecinos utilizados en la estimación de radiancia.

En la primera serie (figuras 3.1 a 3.4), se fija el número de vecinos en 10 y se incrementa progresivamente la cantidad de fotones lanzados. Se observa una mejora clara en la calidad de la imagen: con pocos fotones (1 K o 10 K) la iluminación es incompleta y aparecen artefactos, mientras que a partir de 100 K y especialmente con 1 M, la imagen se estabiliza y los efectos de iluminación global se representan de forma más realista.

En la segunda serie (figuras 3.5 a 3.9), se fija el número de fotones en 100 K y se varía el número de vecinos. Con un solo vecino (fig. 3.5), el resultado es extremadamente ruidoso. A medida que se incrementa el número de vecinos, el ruido se reduce, pero también se pierde detalle. Con 500 vecinos (fig. 3.9), la imagen es más suave, pero aparecen zonas sobredifuminadas. Esto evidencia la necesidad de equilibrar ambos parámetros para obtener una imagen precisa y con bajo ruido.

3.2. Cornell box con bolas difusas

Primero se fijará un número de vecinos para la estimación de radiancia, $k=10$.

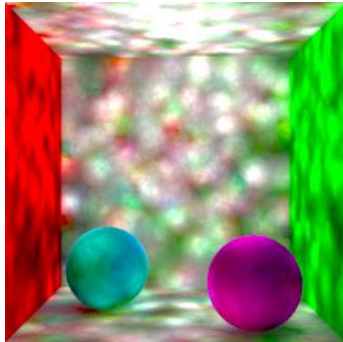


Figura 3.10 - 1K fotones. 10 vecinos

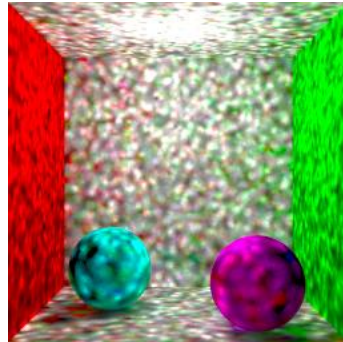


Figura 3.11 - 10K fotones. 10 vecinos

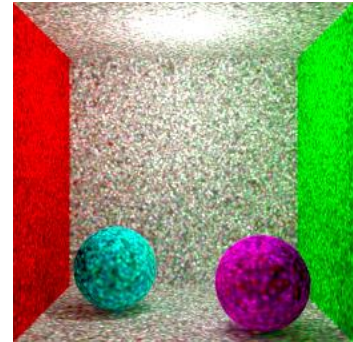


Figura 3.12 - 100K fotones. 10 vecinos

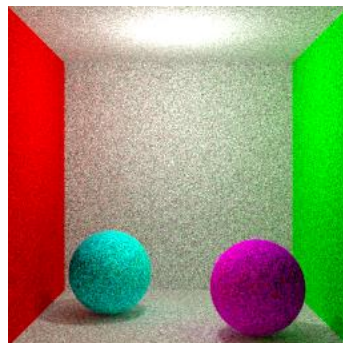


Figura 3.13 - 1M fotones. 10 vecinos

Segundo, fijaremos un número de fotones, $N=100K$.

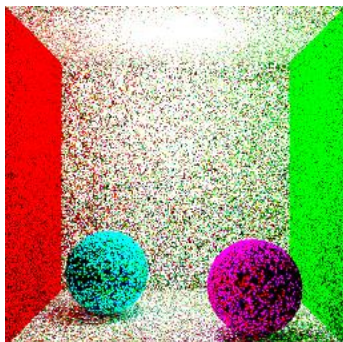


Figura 3.14 - 100K fotones. 1 vecino

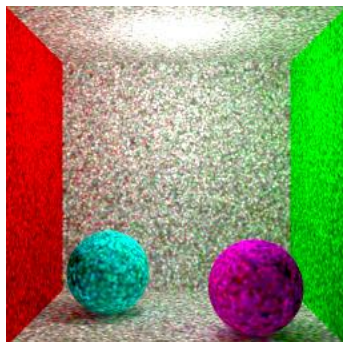


Figura 3.15 - 100K fotones. 10 vecinos

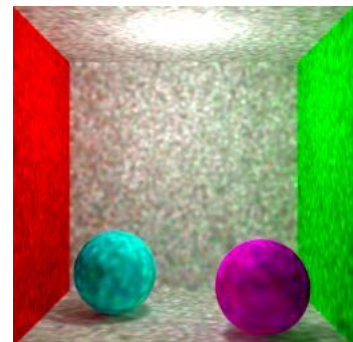


Figura 3.16 - 100K fotones. 50 vecinos

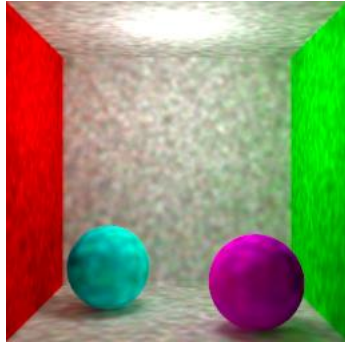


Figura 3.17 - 100K fotones.
100 vecinos

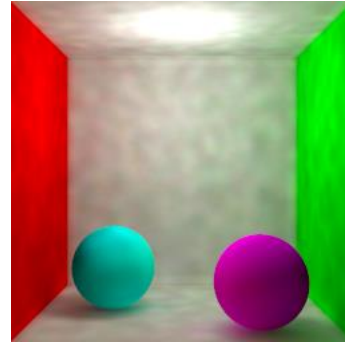


Figura 3.18 - 100K fotones.
500 vecinos

Esta serie de imágenes muestra cómo influye el número de fotones emitidos y el número de vecinos utilizados en la calidad de una escena con superficies completamente difusas. Con pocos fotones (fig. 3.10 a 3.11), la iluminación es ruidosa y los efectos globales son poco precisos. A medida que se incrementa el número de fotones (hasta 1 M en la fig. 3.13), la distribución de energía es más realista, mostrando sombras suaves y color bleeding bien definido.

Por otro lado, al mantener constante el número de fotones (100K) y variar los vecinos (fig. 3.14 a 3.18), se observa una progresiva reducción del ruido. No obstante, al llegar a 500 vecinos (fig. 3.18), se pierde nitidez y los bordes se difuminan. En escenas difusas como esta, es crucial balancear ambos parámetros para obtener resultados precisos sin sacrificar detalle.

3.3. Convergencia del algoritmo

En cuanto a la escena del apartado 3.1, estos son los resultados:

Fotones	Vecinos	Tiempo
1K	10	3.25125s
10K	10	3.58055s
100K	10	6.77039s
1M	10	16.184s
100K	1	2.71083s
100K	50	13.6814s
100K	100	30.3112s
100K	500	2m 6.61105s

En cuanto a la escena del apartado 3.2, estos son los resultados:

Fotones	Vecinos	Tiempo
1K	10	3.12385s
10K	10	3.48261s
100K	10	4.73305s
1M	10	17.6826s
100K	1	3.12058s
100K	50	14.3584s
100K	100	32.4954s
100K	500	2m 47.6066s

4. Extensiones Implementadas

4.1. Otros kernels – kernel gaussiano

En este trabajo se ha implementado un kernel gaussiano para suavizar la estimación de radiancia a partir de los fotones cercanos. En lugar de considerar todos los vecinos con el mismo peso (como haría un kernel uniforme), el kernel gaussiano da mayor importancia a los fotones más cercanos al punto de intersección, atenuando progresivamente la influencia de los más alejados.

$$L_r(x, \vec{\omega}) \approx \sum_{p=1}^N f_r(x, \vec{\omega}_i, \vec{\omega}_0) \Delta\Phi_p(x, \vec{\omega}_p) w_{pg}$$

$$w_{pg} = \alpha \left[1 - \frac{1 - e^{-\beta \frac{d_p^2}{2r^2}}}{1 - e^{-\beta}} \right]$$

Donde:

- w_{pg} es el peso del fotón
- $\alpha = 0.918$ [1]
- $\beta = 1.953$ [1]
- d_p es la distancia entre el fotón y el punto de intersección.

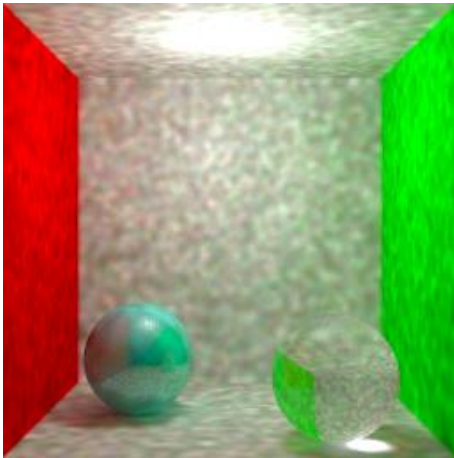


Figura 4.2 - 100K fotones. 100 vecinos.
kernel gaussiano

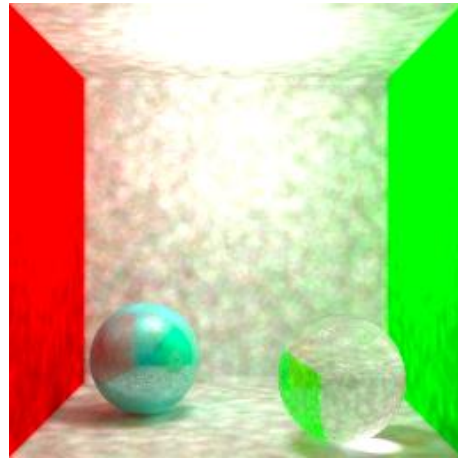


Figura 4.2 - 100K fotones. 100 vecinos.
Kernel constante

5. Referencias

- [Ray Tracing in a Weekend.pdf](#)
- [Ray Tracing in One Weekend](#)
- <https://youtube.com/playlist?list=PLVotA8ycjnCuOGBTriHdw80lpr1r33IM&si=bBXvteJsRxzSEa91>
- [Path tracing - Wikipedia](#)
- [1] [paper.pdf](#)
- Material de clase